期中考範例考題：解答與詳解

對應 midterm_sample.md。先自己寫完再看；每題的詳解都寫出「為什麼」，正式考試計算題要有同樣的過程才有分。

一、選擇題

題	答案	為什麼
1	B	`C0 AF` 是 overlong：它想表示 U+002F（`/`），但 U+002F 只要 1 byte（`2F`）就能表示；UTF-8 規定一個 code point 只能用**最短**的形式，所以 `C0`、`C1` 開頭的序列永遠不合法。(A) 是「多」，(C) 是 😜（U+1F600，4 bytes），(D) 是 `A` 加換行。
2	B	`EF BB BF` 是 U+FEFF 的 UTF-8，叫 BOM（byte order mark）。本課程的規則：檔頭有 BOM 就跳過，不算內容。
3	B	8 000 sample / 秒 × 2 bytes × 1 聲道 × 10 秒 = 160 000 bytes。
4	C	f_s = 8 000 時 Nyquist frequency 是 4 000 Hz；5 000 Hz 超過了，會被「摺」回 |5 000 − 8 000| = 3 000 Hz（第 3 週 `alias_demo.py`：兩個波形只差正負號）。
5	B	Shannon：任何無失真編碼 L ≥ H；Huffman 是最佳的 prefix code，而且 L < H + 1。只有一種符號時是例外（H = 0 但 L = 1）。
6	B	沒有任何 code 是另一個 code 的前綴，位元流才能不用分隔符號就唯一解碼。
7	B	TCP 是 byte stream：一次 `send` 的資料可能被拆成幾次 `recv` 到（半包），幾次 `send` 也可能一次 `recv` 到（黏包）。長度前綴讓接收端知道「這一則有幾個 bytes」。遺失由 TCP 自己重傳，不是 frame 的工作。
8	C	WAV（RIFF）的欄位是 little-endian（Intel 的習慣）；網路協定的慣例是 big-endian（network byte order），Team 1 的 frame 長度照這個慣例。
9	A	主瓣寬度 ∝ f_s / P。N 只是「畫幾根」（zero padding 是內插），M 決定時間解析度。
10	B	DFT 把 frame 當成週期訊號，rectangular 切出來的兩端接不起來，不連續會漏到所有頻率；Hamming 把兩端壓平。

二、計算題

題 1｜UTF-8 與符號統計（MP1）

(a) 先把每個字元換成 bytes：

字元	code point	UTF-8 bytes	幾 bytes
BOM	U+FEFF	`EF BB BF`	3
多	U+591A	`E5 A4 9A`	3
媒	U+5A92	`E5 AA 92`	3
體	U+9AD4	`E9 AB 94`	3
空白	U+0020	`20`	1
O	U+004F	`4F`	1
K	U+004B	`4B`	1
`\n`	U+000A	`0A`	1

合計 3 + 9 + 1 + 1 + 1 + 1 = **16 bytes**。

(b) BOM 跳過不算。符號是：多、媒、體、空白、O、K、`\n` → **7 個符號，7 種**（每種各 1 次，機率都是 1/7）。 MP1 的輸出裡 `\n` 會寫成 `"\n"`，空白寫成 `" "`。

(c) U+591A 的二進位：5 = 0101、9 = 1001、1 = 0001、A = 1010 → `0101 1001 0001 1010`，共 16 bits。 U+0800 ≤ U+591A ≤ U+FFFF，用三 byte 格式 `1110xxxx 10xxxxxx 10xxxxxx`，三個 x 區共 4 + 6 + 6 = 16 bits，剛好塞進去：

```
0101 100100 011010
│    │      └─ 10 011010 = 1001 1010 = 9A
│    └──────── 10 100100 = 1010 0100 = A4
└───────────── 1110 0101           = E5
```

答：`E5 A4 9A`。

題 2｜取樣與量化（MP2）

(a) 資料：2 s × 8 000 sample/s × 2 bytes = 32 000 bytes；加 44 bytes 檔頭 = **32 044 bytes**。

(b) 均勻量化、誤差當成均勻分布的雜訊，功率 $\Delta^2 / 12$；滿刻度弦波的功率是 $A^2 / 2$。兩者相除取 dB：

SQNR ≈ **6.02·n + 1.76 dB**，n = 16 → 96.32 + 1.76 = **98.1 dB**。（`wavegen.py` 量到 98.16 dB。）只寫 6.02·n = 96.3 dB 的是「峰值比」的定義（2^n 的 dB），差 1.76 dB，考試要說清楚用哪一個。

(c) 振幅減半 → 訊號功率變 1/4 → 少 $10 \cdot \log_{10} 4$ = **6.02 dB** → 約 **92.1 dB**。 量化雜訊的功率只和階距 Δ 有關，Δ 沒變，所以雜訊不變、只有訊號變小。這就是為什麼錄音太小聲會「浪費 bit depth」。

(d) 8-bit、full scale = 127：q = rint(0.30 × 127) = rint(38.1) = **38**。 $\hat{x} = 38 / 127 = 0.29921\ldots$；誤差 $x - \hat{x} = 0.30 - 0.2992 \approx +0.0008$（小於 $\Delta/2 = 1/254 \approx 0.0039$，合理）。

題 3｜定長編碼與 bit packing（MP3）

(a) 符號種類 90 + 1（EOF）= 91。要 $2^b \geq 91$：$2^6 = 64$ 不夠、$2^7 = 128$ 夠 → **7 bits**。（MP3 的參考實作固定用 7 bits，所以符號超過 128 種會報錯。）

(b) 編 1 000 個字元 + 1 個 EOF，每個 7 bits：1 001 × 7 = **7 007 bits**。補 0 到整數 byte：7 007 ÷ 8 = 875.875 → **876 bytes**（最後一個 byte 補 1 個 0）。

(c) 876 ÷ 1 800 ≈ **0.487**（48.7%）。本課程壓縮率的定義是「壓縮後 ÷ 原始」，越小越好。注意這還沒算 codebook.csv；真的要比較得把 codebook 也算進去（Team 1 的 wire_bytes 就是這樣算）。

題 4｜熵與 Huffman（MP4、Team 1）

(a) N = 8；p(A) = 4/8 = 1/2、p(B) = 2/8 = 1/4、p(C) = p(D) = 1/8。

$H = \Sigma\, p \cdot \log_2(1/p) = \frac{1}{2}\cdot 1 + \frac{1}{4}\cdot 2 + \frac{1}{8}\cdot 3 + \frac{1}{8}\cdot 3 = 0.5 + 0.5 + 0.375 + 0.375 =$ **1.75 bits / 符號**。

(b) 佇列依次數由小到大：[C:1][D:1][B:2][A:4]。

輪	取出最小的兩個	新節點	佇列
1	C:1、D:1	CD:2	[B:2][CD:2][A:4]（B 與 CD 平手：這裡讓先進佇列的 B 排前面；平手規則可以自訂，編碼端與解碼端一致即可）
2	B:2、CD:2	BCD:4	[A:4][BCD:4]
3	A:4、BCD:4	根:8	—

```
      (8)
    0/   \1
   A     (4)
       0/   \1
       B    (2)
          0/   \1
          C     D
```

code：**A = 0、B = 10、C = 110、D = 111**。（0/1 左右互換、或平手時先合併 B 與 C 之類的其他合法樹也算對，只要是 prefix code 而且長度是 1、2、3、3。）

(c) 總 bits = 4×1 + 2×2 + 1×3 + 1×3 = **14 bits**；L = 14 / 8 = **1.75 bits / 符號**。H = 1.75 ≤ L = 1.75 < 2.75 ✓。這一題 L 剛好等於 H，因為每個機率都是 2 的負次方（code 長度 = $\log_2(1/p)$ 剛好是整數）；一般情況 L 會比 H 大一點。定長編碼要 2 bits × 8 = 16 bits，Huffman 省了 2 bits。

題 5｜STFT 參數（MP5）

(a) 毫秒換 sample：乘 f_s 再除 1 000。P = 16 000 × 25 / 1 000 = **400**；N = 16 000 × 32 / 1 000 = **512**；M = 16 000 × 10 / 1 000 = **160**。P = 400 < N = 512：每個 frame 後面補 112 個 0（zero padding）。

(b) bin 間距 = f_s / N = 16 000 / 512 = **31.25 Hz**。第 k 根的頻率 = k·f_s / N → k = 100 是 **3 125 Hz**。（注意兩個「解析度」：bin 間距由 N 決定，是畫幾根；能不能「分辨」兩個相近的頻率由 P 決定，主瓣寬的量級是 f_s / P（16 000 / 400 = 40 Hz），Hamming 再寬一倍。選擇第 9 題問的是後者。）

(c) 直接算 DFT：每根 N 次複數乘法、N 根 → $N^2 = 512^2 =$ **262 144 次**。FFT（radix-2）：$(N / 2)\cdot\log_2 N = 256 \times 9 =$ **2 304 次**，約 114 倍的差距。（寫 $N\cdot\log_2 N = 4\,608$ 也接受，只要說明是哪一種算法。）補充：1 秒的語音有 $\lfloor(16\,000 - 400) / 160\rfloor + 1 = 98$ 個 frame（最後不足一個 frame 的不算，MP5 的參考答案也是這樣）。

三、簡答題（給分要點）

1. side information 與 codebook。 - **Side information（附帶資訊）**：解碼端要把資料還原**所必須知道、但本身不是資料內容**的一切。它不是原檔的一部分，卻必須跟著壓縮資料一起送（或雙方事先約定好），否則收到的位元流只是一堆 0 和 1。 - **Codebook（碼簿）**：符號 ↔ code 的對照表。Huffman 的 codebook 是**依資料算出來的**（每個檔案的次數表不同，樹就不同），所以解碼端沒有它就無法解碼；它是 side information 裡**最大、也最重要的一項**。 - **關係**：codebook ⊂ side information。side information 還包括其他「怎麼解」的資訊：符號種類（byte / UTF-8 字元 / 16-bit sample）、原始長度、符號數、最後一個 byte 補了幾個 0、WAV 的 head / tail chunk 等。相對地，MP3 的定長編碼如果雙方約定好「依次數排序、7 bits」，codebook 就幾乎不用傳——side information 可以用「約定」換掉一部分，但 Huffman 不行。 - **以 Team 1 的區塊為例**：`kind`（符號種類）、`orig_len`（原始長度）、`n_syms`（符號數）、`K`（符號種類數）、`codebook`（K 個符號與各自的 code 或 code 長度）、S16 的 head / tail chunk → 全部是 side information；只有 `bitstream` 是真正的資料。 - **為什麼要算進壓縮率**：對方收到的 bytes（wire_bytes）包含 side information，少了它就解不開，所以它是「為了傳這個檔案真正付出的成本」。小檔案或符號種類多（中文字元當符號，K 可達數千）時，codebook 可能比 bitstream 省下來的還多，壓縮率會 > 1——Team 1 報告「實際壓縮率與純編碼的壓縮率差多少」那一題要討論的就是這個。只算 bitstream 的壓縮率是自欺。

給分：定義各 1 分、關係（codebook 是 side information 的一部分）1 分、區塊欄位分類 1 分、為什麼算進壓縮率 1 分。

2. frame 的 bytes。 length = type（1）+ payload（3）= 4，4 bytes big-endian → `00 00 00 04`；type `01`；payload `E5 97 A8`：

```
00 00 00 04 | 01 | E5 97 A8        共 8 bytes
```

接收端先用 `recv_all` **一定收滿** 4 bytes 讀出長度 4，再 `recv_all` **剛好** 4 bytes，不多不少；下一則從下一個 byte 開始。沒有長度前綴的話，TCP 可能把兩則訊息黏在一起送到、或把一則拆成兩次送到，接收端分不出界線。

3. 符號的定義。 中文一個字 3 bytes。以 **byte** 為符號：三個 byte 的分布都很平（前導 byte 集中在 E4–E9，後兩個 byte 分散），熵高（第 3 週 1.7 的中英文範例：6.01 bits / byte → 最多只能壓到 75%）。以 **UTF-8 字元**為符號：常用字集中、罕用字少，同一個範例 5.63 bits / 字元，而一個中文字原本是 24 bits → 可以壓到 46%。但字元為符號時 codebook 也大得多（K 可達數千種），檔案小的時候可能得不償失——這就是第 1 題 side information 的成本；報告要兩個一起看。

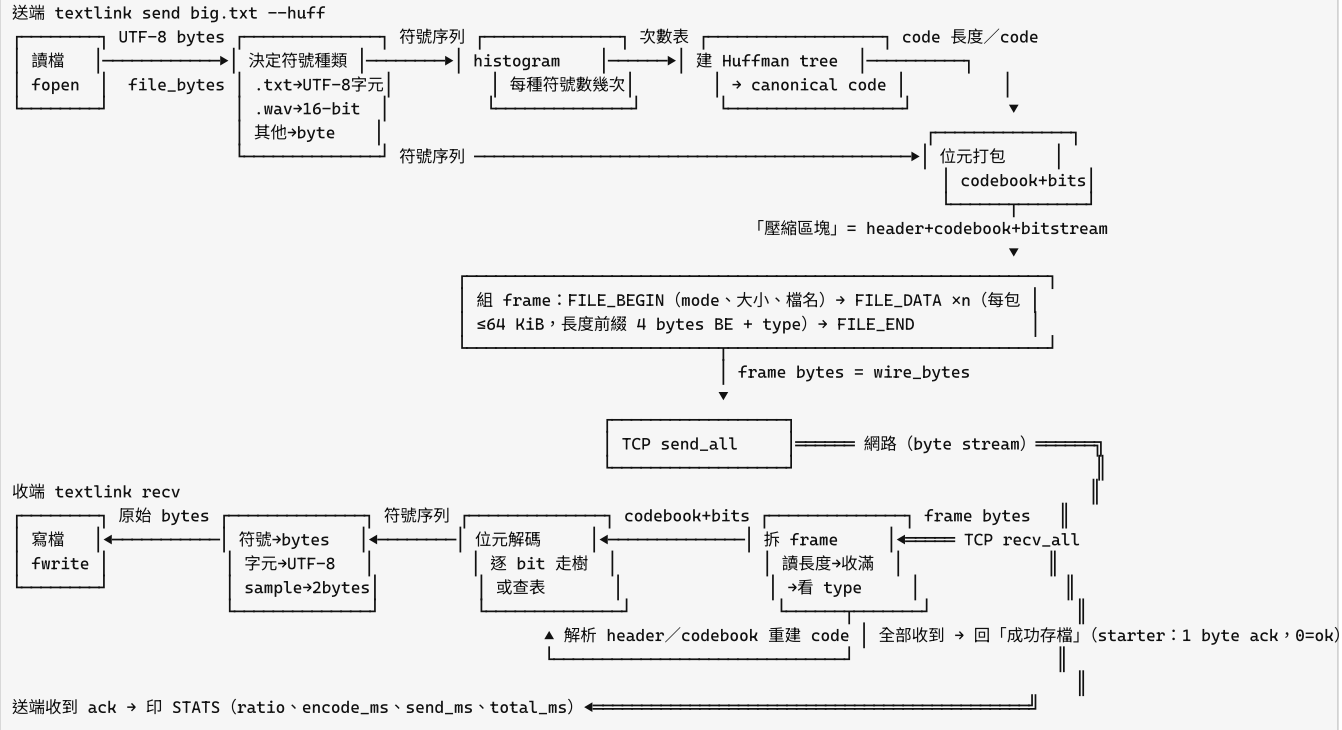
4. 小檔案 Huffman 反而慢。 可能原因（任兩個）：(1) 編碼時間（建 histogram、建樹、位元打包）比傳輸時間長；(2) codebook 開銷讓 wire_bytes 反而比原檔大；(3) 小檔案時固定成本主導：TCP 建立連線、等對方回覆，這些時間和檔案大小無關；(4) 解碼端的 decode_ms。怎麼分清楚：STATS 要**分開**記 encode_ms、send_ms、decode_ms、wire_bytes、file_bytes；同一個檔案跑至少 5 次取中位數（規格的要求）；用不同大小的檔案（1 KB → 10 MB）畫「檔案大小 vs 時間」看壓縮與不壓縮的交叉點；比較 `--raw` 與 `--huff` 的 send_ms 就能把編碼時間與傳輸時間分開。

5. zero padding。 DFT 的結果等於「原本 P 點訊號的頻譜（DTFT）」在 N 個點上取樣。補 0 只是讓取樣點變密（從 P 根變 N 根），多出來的根是原頻譜的**內插**，不增加任何資訊；主瓣寬度仍由 P 決定（$\propto$ f_s / P），兩個差距小於主瓣寬的頻率還是分不開。要提高解析度要加長 P（視窗長）；代價是時間解析度變差：frame 太長，裡面的語音已經在變（pitch、formant 在移動），spectrogram 的時間軸糊掉，而且寬頻 spectrogram 裡的 pitch pulse 會看不見。

6. leakage 與窗函數。 DFT 隱含假設這 256 點是週期為 256 的週期訊號。170 Hz 在 256 點裡有 256 / 8 000 × 170 = 5.44 個週期，不是整數，所以頭尾接不上，接縫處有一個「跳躍」。跳躍不是原訊號有的成分，它的能量散到所有頻率，所以本來該是一根的頻譜變成主瓣加一串 side lobes（rectangular 的最高 side lobe 約 **−13 dB**，而且衰減得慢）。 Hamming window 把 frame 兩端平滑地壓到接近 0，接縫不再跳，side lobe 降到約 **−41 dB**；代價是主瓣寬約 **2 倍**（頻率解析度變差）。語音分析慣用 Hamming，因為語音的 harmonics 振幅差好幾十 dB，side lobe 太高會把弱的 harmonic 蓋掉。

四、系統題：TextLink 端對端壓縮傳輸

(a) block diagram（參考圖；方塊名稱可以不同，功能對、資料流標對就給分）



給分要點（6 分）：送端 6 個方塊（讀檔、符號、histogram、建樹 / codebook、打包、組 frame）+ TCP；收端對稱 5 個方塊 + ack；每個箭頭有資料型態（bytes / 符號序列 / 次數表 / codebook + bitstream / frame bytes）。缺 codebook 傳給對方、缺長度前綴、缺 ack 各扣 1 分。

(b) `file_bytes` 在「讀檔 → 決定符號種類」的箭頭（原檔的 bytes 數）；`wire_bytes` 在「組 frame → TCP」的箭頭（所有 FILE_BEGIN / FILE_DATA / FILE_END frame 的 bytes 總和，含 header 與 codebook）。 `encode_ms` 涵蓋符號切分 → histogram → 建樹 → 打包；`send_ms` 涵蓋組 frame → TCP 送出到對方回 ack；`decode_ms` 涵蓋收端的解析 codebook → 位元解碼 → 還原 bytes。

(c) 整條管線**都應該是無失真的**：Huffman 是無失真，frame 與 TCP 不改內容。兩個要注意的邊界：(1) 以 UTF-8 字元為符號時，遇到不合法的 UTF-8 bytes 不能「修正」它（那就失真了），規格規定退回以 byte 為符號；(2) 以 16-bit sample 為符號時，WAV 的 `data` 以外的 chunk（檔頭、LIST）要原樣存在區塊裡（head / tail bytes），`data` 長度是奇數時最後一個 byte 也要保留——否則還原出來的檔案和原檔不同。驗收是「逐 byte 相同」，所以任何一個方塊有失真都會被抓到。

(d) TCP send_all / recv_all 換成 `sendto` / `recvfrom`；「組 frame」改成「每 20 ms 一包的封包（header：seq、timestamp、ssrc）」；收端新增 **jitter buffer**（依 seq 排序、固定節奏取出）與 **遺失補償 PLC**；ack 拿掉（即時語音不等重傳），改成結束時的 BYE 與 STATS。（任三個即可。）

(e) source coding 與 channel coding。

	Source coding（信源編碼）	Channel coding（通道編碼）
目的	**拿掉**資料裡的 redundancy，用最少的 bits 表示（壓縮）	**加上**受控的 redundancy，讓通道造成的錯誤能被偵測或更正
依據	資料的統計特性（符號的機率、熵）	通道的錯誤特性（錯誤率、是叢發還是隨機）
在圖上的位置	兩端的**最外側**：送端「histogram → 建樹 → 位元打包」、收端「位元解碼 → 還原」——Huffman 就是 source coding（entropy coding）	**最靠近通道**：在 TextLink 的方塊裡**沒有**，它在 TCP 以下的層：TCP 的 checksum（偵錯）+ 重傳（ARQ，用重送當作「補救」）、乙太網路 / Wi-Fi 的 CRC（偵錯）、實體層的 FEC（convolutional / LDPC code，更正錯誤）
TextLink 的例子	`--huff`：wire_bytes 變小	沒有自己做；所以「收到的 bytes 一定正確」這件事是**TCP 保證的**，不是 Huffman 保證的；換成 UDP（Team 2）就沒有重傳，錯誤要自己面對（PLC）

為什麼目的相反卻能分開設計：Shannon 的 **source–channel separation**——先把資料壓到接近熵（source coding），再依通道的容量獨立加上保護（channel coding），兩段分開最佳化，整體仍可逼近最佳，不需要一起設計。這就是為什麼多媒體課只管壓縮、通訊課只管編碼，而兩者能拼在同一條管線上（本課程 1.1 的管線圖：壓縮在應用層、保護在傳輸與實體層）。

給分：兩個定義各 1 分（要講到 redundancy 的拿掉 / 加上）；位置 1 分（source 在兩端、channel 在 TCP 以下 / 靠近通道，並指出 TextLink 自己沒做）；separation 的一句話是加分項，寫錯不扣分。